Recap: closed, open-weight, open-source

"Open" is used loosely. What matters is what you actually get to download.

you get: an API	+ the trained weights	+ training code and data

Closed

GPT · Claude · Gemini

- ✓ an API and a chat app
- ✗ the model — it runs on their servers, and your data goes there
- ✗ control — versions change or get retired under you

Open-weight

Qwen · DeepSeek · Kimi · GLM · Mistral · Gemma · gpt-oss · Llama

- ✓ the trained weights — run, quantize, fine-tune anywhere
- ✗ training data and code — you can run it, not rebuild it

Licences vary: Apache 2.0 / MIT, or custom ones with restrictions (Llama, Gemma).

Open-source

OLMo (Ai2) · Pythia (EleutherAI)

- ✓ weights + training code + data — reproducible from scratch
- ✗ rare, and behind the best open-weight models

Most "open-source LLMs" in the news are really open-weight.

Why run open models

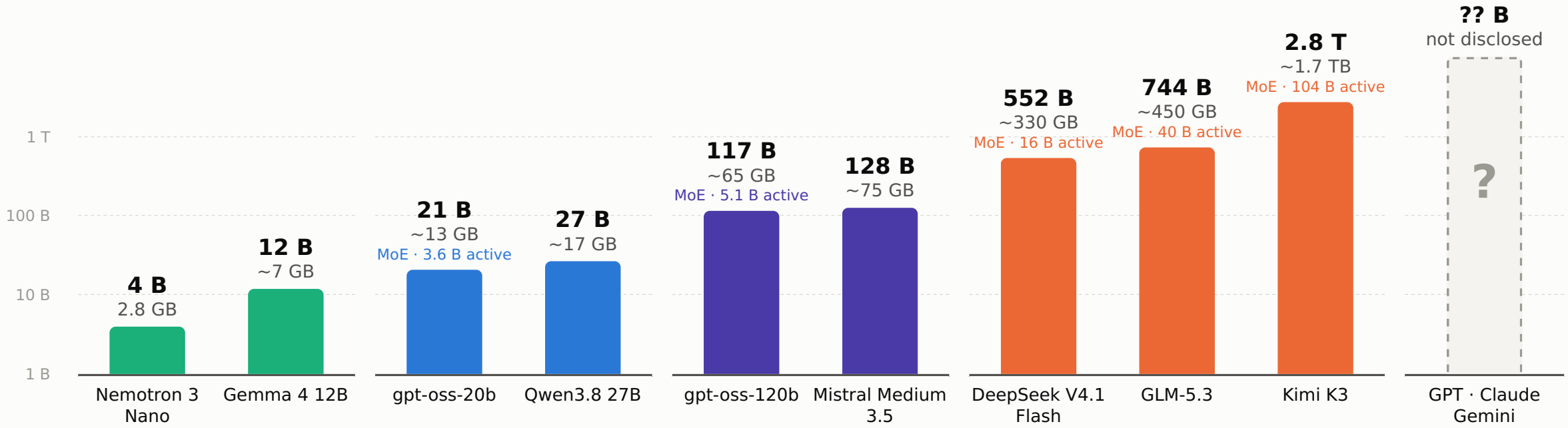
- data never leaves your machine
- projects under **NDA** that forbid cloud AI
- fixed version → reproducible results
- no price per token
- works offline and on HPC nodes

The cost

- you need the hardware
- you do the setup and the updates
- usually behind the best closed models

What runs where

The whole model has to fit into memory — RAM, GPU memory or unified memory. Parameters on a log scale, size at 4-bit quantization.



Phone · laptop
no GPU, 8–16 GB RAM
slow, but private

Gaming GPU
RTX 4090 / 5090, 24–32 GB
or a Mac with 32–64 GB

Pro GPU · DGX Spark
RTX PRO 6000, 96 GB
DGX Spark, 128 GB unified

Datacentre · DGX B300
8 GPUs, ~2.3 TB of GPU memory in one node — you use these over an API

Closed
their datacentres only

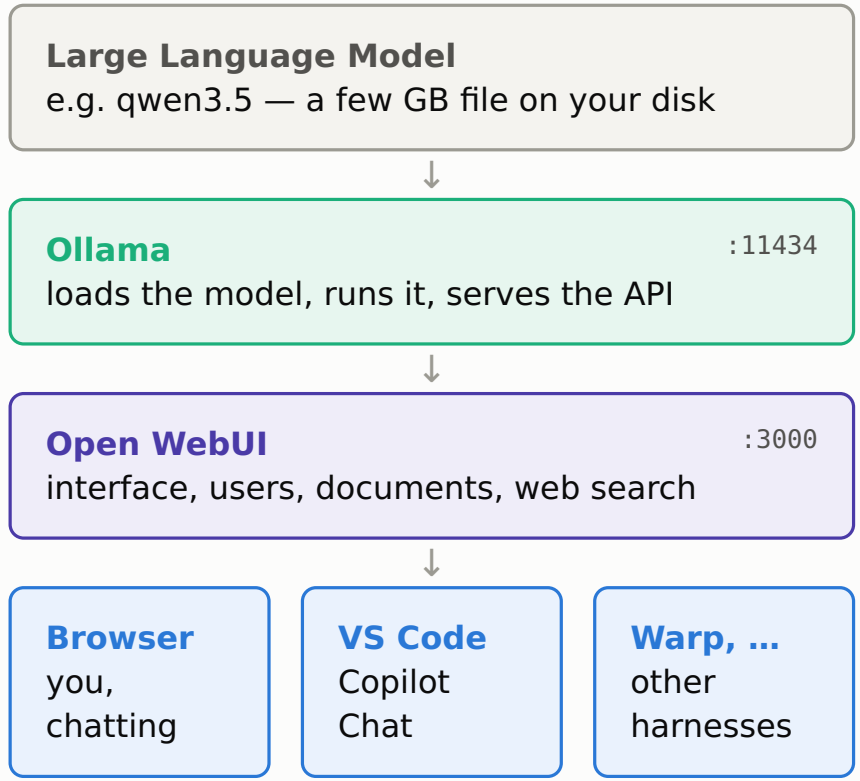
Rule of thumb: ~0.6 GB per billion parameters at 4-bit, plus a few GB for the context. **MoE:** all parameters must fit in memory, but only the active ones set the speed.

Running it yourself: Ollama + Open WebUI

Two free tools turn a downloaded model into your own ChatGPT.

- **Ollama** — downloads and runs open-weight models, on CPU, NVIDIA, AMD or Apple silicon
  - install — Linux: `curl -fsSL https://ollama.com/install.sh | sh`
    macOS / Windows: installer from **ollama.com/download**
  - `ollama pull qwen3.5` → `ollama run qwen3.5`
  - models, already quantized: **ollama.com/search**
  - API on `localhost:11434` for Python, VS Code, Open WebUI

- **Open WebUI** — a ChatGPT-style web interface on top of Ollama
  - chat history, documents, web search, multiple users
  - runs as a **Docker** container (or `pip install open-webui`)
  - point it at Ollama, open `localhost:3000` in the browser



VS Code can also skip Open WebUI and talk to Ollama directly (Ollama extension).

eInfra models in VS Code

eInfra runs the big open-weight models for Czech academia. Copilot can use them as a *custom endpoint*.

1. **Get an API key:** `chat.ai.e-infra.cz` → Settings → Account → API Keys

2. **Add the endpoint:** Copilot Chat → model picker → *Manage Models* → *Add Models…* → **Custom Endpoint**; name it `eInfra`, paste the key

3. **Fill in the models:** VS Code opens `chatLanguageModels.json` — keep its `apiKey` line, add one entry per model (right)

4. **Use it:** the models appear on their own **eInfra** tab in the model picker

- **`id`** must match the server exactly — list them at `llm.ai.e-infra.cz/v1/models` (with your key)
- **`vision`, `thinking`, token limits** — from the model card; too high a limit and long chats fail
- **Status** of all models: `llm.ai.e-infra.cz/status`
- **Your usage:** `llm.ai.e-infra.cz/usage`

```
[
  {
    "name": "eInfra",
    "vendor": "customendpoint",
    "apiKey": "${input:chat.lm.secret.…}",
    "apiType": "chat-completions",
    "models": [
      {
        "id": "glm-5.3",
        "name": "GLM-5.3 (eInfra)",
        "url": "https://llm.ai.e-infra.cz/v1",
        "toolCalling": true,
        "vision": false,
        "thinking": true,
        "contextWindow": 1048576,
        "maxOutputTokens": 32000
      },
      {
        "id": "kimi-k3",
        "name": "Kimi K3 (eInfra)",
        "url": "https://llm.ai.e-infra.cz/v1",
        "toolCalling": true,
        "vision": true,
        "thinking": true,
        "contextWindow": 1048576,
        "maxOutputTokens": 32000
      }
    ]
  }
]
```

← user-specific: keep the `apiKey` line VS Code generated for you